Travail pratique 02 - Énoncé

Objectif

L'objectif de ce travail est de **valider l'atteinte des sous-objectifs** dans le cadre du cours :

1. Effectuer le développement d'applications Web transactionnelles. (Compétence 00SU)
   - 5 - Programmer la logique applicative du côté serveur
   - 6 - Programmer la logique applicative du côté client
   - 7 - Contrôler la qualité de l'application
   - 9 - Rédiger la documentation

Mise en situation

À partir de **récits utilisateurs**, et en suivant cette **méthodologie**, créer un **jeu simple en Vue.js** dans l'univers **Pokémon** dont l'objectif est de **survivre à cinq combats de dresseurs** tout en obtenant le plus de **Pokédollars** possible.

L'application communique avec un service Web fourni.

***IMPORTANT :** Le jeu peut être adapté pour un autre sujet**. Le jeu doit rester** simple**, garder le même type de progression. Le joueur choisit** cinq personnages **parmi un** choix prédéfini **pour former son équipe et peut changer de personnage au besoin,** une fois par tour**.*

Détails supplémentaires

1. Respecter les bonnes pratiques de code suivantes
2. Vous devez respecter les récits utilisateurs fournis.
3. Vous devez respecter les consignes énoncées plus bas.

4. Le jeu doit rester simple : concentrez-vous sur la réalisation des récits utilisateurs, sans ajouter de mécaniques complexes qui modifieraient l'esprit du travail.
5. Dans un fichier **README.md à la racine**, vous devez identifier :
   - Description du jeu
   - Votre position en tant qu'équipe sur l'utilisation de l'IA. Vous devez être clair sur ce que vous décidez de mettre comme règle.
   - URL du repo GitHub (privé)
   - URL de la documentation déployée

Récits utilisateurs

🟢 = Récit disponible à tout moment.
🟡 = Récit disponible dans la page d'accueil.
🔴 = Récit disponible en cours de partie.
P$ = Pokédollars

	En tant qu'utilisateur	Précisions
🟢 1	Je veux pouvoir accéder à la page d'accueil.	Un utilisateur peut à tout moment revenir à la page d'accueil. À noter que s'il a commencé une partie, il perdra la partie en cours après avoir confirmé qu'il veut quitter.
🟢 2	Je veux pouvoir voir le tableau de pointage.	Le tableau contient tous les noms des dresseurs gagnants avec le pointage (P$ gagnés). Ce tableau est trié en fonction du nombre de P$ gagnés (en tête de liste = le dresseur avec le plus de P$). Un utilisateur peut à tout moment voir la page de pointages. À noter que s'il a commencé une partie, il perdra la partie en cours après avoir confirmé qu'il veut quitter.
🟡 3	Je veux pouvoir choisir le nom de mon dresseur.	Le nom est au choix du joueur et est obligatoire.
🟡 4	Je veux pouvoir choisir mes Pokémon de départ.	Le choix des Pokémon se fait à partir d'une liste prédéfinie (avec API). Le joueur doit sélectionner **5 Pokémon** pour former son équipe de départ.

	En tant qu'utilisateur	Précisions
🟡 5	Je veux commencer une partie.	
🔴 6	Je veux pouvoir voir les caractéristiques associées à mon personnage.	Les caractéristiques sont le nom du dresseur, son expérience (tous les joueurs sont "Maitre"), le nombre de P$ gagnés (0 en début de partie), son équipe de 5 Pokémon, le nom du Pokémon actif et sa vie en pourcentage (100 % en début de partie).
🔴 7	Je veux pouvoir voir les caractéristiques de mon ennemi.	Les caractéristiques sont le nom du Pokémon adverse, son expérience (débutant, confirmé, expert ou maitre), le nombre de P$ qu'il rapporte, son type si disponible et sa vie en pourcentage. Dans l'API REST, 1 = débutant et 4 = Maitre.
🔴 8	Je veux voir quel est mon combat en cours.	Au démarrage, le combat de dresseurs en cours est 1/5.
🔴 9	Je veux combattre un ennemi.	Dans un combat, le Pokémon actif du joueur et le Pokémon adverse s'affrontent dans une mécanique simple inspirée d'un combat Pokémon au tour par tour. Un personnage a plus de probabilité de toucher la cible s'il possède plus d'expérience (débutant = 20 %, confirmé = 35 %, expert = 50 %, maitre = 70 %). Un Pokémon touché fait perdre aléatoirement de 3 % à 6 % de vie. Le joueur peut aussi changer de Pokémon actif au besoin, **une fois par tour**, à partir de son équipe. Il est autorisé de modifier tous les pourcentages ci-dessus afin d'avoir une meilleure expérience de jeu. Si un Pokémon adverse est mis K.O., le joueur gagne les P$ (récit #10) et le combat est terminé (récit #11). Si tous les Pokémon du joueur sont mis K.O., c'est la fin de la partie (récit #13). Un Pokémon adverse vaincu ne peut plus combattre à nouveau.
🔴 10	Je veux pouvoir récupérer les P$ de l'ennemi.	Le joueur gagne tous les P$ de l'ennemi s'il met K.O., lors d'un combat (récit #9), le Pokémon adverse. Un message doit informer l'utilisateur des P$ gagnés.
🔴 11	Je veux pouvoir terminer le combat en cours.	Le fait de terminer un combat affiche un nouveau Pokémon adverse et incrémente "combat en cours".

	En tant qu'utilisateur	Précisions
🔴 12	Je peux gagner une partie.	Si l'utilisateur a survécu à 5 combats de dresseurs, il est informé de son pointage (nombre de P$ gagnés) et il est ensuite redirigé vers la page des pointages. Le nom du joueur et son pointage doivent se retrouver dans le tableau de pointage (récit #14).
🔴 13	Je peux perdre une partie.	Le joueur perd la partie dès que tous les Pokémon de son équipe ont une vie de 0 ou moins. Un message doit informer l'utilisateur qu'il a perdu (il n'a pas complété l'objectif de 5 combats de dresseurs). Il est ensuite redirigé vers la page d'accueil.
🔴 14	Je veux ajouter mon nom au tableau des pointages	Un joueur ayant réussi 5 combats de dresseurs doit se retrouver dans le tableau des pointages (récit #2).
🔴 15	Je veux pouvoir soigner mon Pokémon.	Un joueur peut décider, en une même action, de terminer le combat (récit #11) et de soigner le Pokémon actif. Le coût de soin est de 5 P$ par % de vie récupéré. Les soins sont effectués avant l'apparition du prochain adversaire.

Consignes

- Travail pratique **en équipe de deux (20 % de la note finale)**
- Réalisé **en dehors** du cours et du temps en classe
- Durée approximative de **40 heures par équipe**
- Le fichier de remise est un fichier compressé nommé : **appweb_trpr02_votrenom.zip**
- Le fichier compressé doit contenir :
  - Les **fichiers complets** de votre site
  - La documentation dans un dossier à part **docs**
- Remis sur **Léa** à la date spécifiée

IMPORTANT : supprimer le dossier **node_modules**

Critères d'évaluation (100 %)

Le travail est évalué selon les critères suivants et est ensuite pondéré par le pourcentage des composants et récits utilisateurs réalisés et fonctionnels.

Réalisation des récits utilisateurs (30 %)

Récits utilisateurs fonctionnels. **2 % pour chaque récit utilisateur fonctionnel.**

Un récit utilisateur est considéré réalisé seulement s'il est accompagné par des tests pertinents et suffisants pour en certifier le bon comportement.

Qualité des tests unitaires (20 %)

- Choix et pertinence des tests
  - États initiaux
  - États lors de modifications
  - États calculés
- Bonne couverture des tests
  - Composants
  - Fonctions
  - Routes

IMPORTANT : Si des récits utilisateurs sont manquants, vous serez automatiquement pénalisé en proportion avec ce qui manque.
-1 % par récit manquant, maximum de -15 %

Qualité du code (20 %)

- Bonnes pratiques de programmation et code propre
- Bonne gestion des erreurs
- Appel de l'API
- Informations obligatoires

- Architecture facilitant la maintenabilité et l'évolution des applications
- Utilisation de **composants** partagés
- Utilisation de **views** pertinentes
- N'utilisez pas de **store** pour ce projet

IMPORTANT : Si des récits utilisateurs sont manquants, vous serez automatiquement pénalisé en proportion avec ce qui manque.
-1 % par récit manquant, maximum de -15 %

Interface utilisateur (10 %)

- Aucun bogue, aucune erreur d'exécution et aucun problème de performance
- Qualité de l'interface : convivialité, notifications à l'utilisateur, alertes, messages de confirmation, etc.

Documentation et revue de code (20 %)

- Le fichier README est correctement rempli avec les informations demandées
- Le code de votre travail pratique est versionné avec GitHub (repo privé) tout au long du projet
    - Ajoutez-moi à votre repo : **masterjim@gmail.com**
- Documentation **VitePress** fonctionnelle et présente dans le dossier **docs** (un dossier à part pour faciliter le déploiement)
- Faite un merge à la fin de chaque séance de travail
- Faire **4 revues de code** (une pour chaque séance / merge)
- Vos revues de code doivent contenir des commentaires pertinents, comme fait lors de l'exercice (voir exemple aussi)
- Les revues de code sont remises sur Léa sous forme de fichiers Markdown (**pour la semaine 1**). ensuite tout sera sur le site de documentation
- La **documentation** est correctement déployée sur **GitHub Page**
    - Dossier **appweb-trpr02**

- **IMPORTANT :** Vous déployez votre documentation VitePress, pas le travail pratique au complet.
  IMPORTANT : Si vous êtes seul, vous faites vous-même une revue de code. Vous devez être critique et précis comme si vous étiez en équipe. Je vais être sévère sur cette introspection.

Respect des consignes (-10 %)

- Les consignes du travail sont respectées

La grille d'évaluation est fournie sur Léa.

Plagiat et utilisation d'IA

Prendre connaissance du document officiel du Cégep concernant le plagiat.

- **Reproduire partiellement ou totalement un travail sans citer la source**
- **Remettre un travail qui ne provient pas de soi, en tout ou en partie**
- **Générer du code sans citer la source est considéré comme du plagiat**

**Vous devez commenter chaque bloc ou l'IA est utilisée.
`// IA: <outil> – <raison>` sur chaque bloc généré.

IMPORTANT: Vous n'abez pas à mentionner l'IA dans les cas suivants:

- Débogage et recherche de solutions
- Amélioration de l'interface utilisateur (CSS / HTML)
- Recherche et documentation

En cas de doute, je peux vous convoquer à mon bureau pour une entrevue de vérification de votre code.

Conséquences en cas de plagiat ou de tricherie

En plus des conséquences normales du Cégep, je me réserve le droit d'empêcher l'étudiant de faire le TP03 en équipe.